

# Deep learning avec Pytorch

Partie 6 : Sur et sous entraînement



---

Présenté par **Morgan Gautherot**



## Train / dev / test sets

---



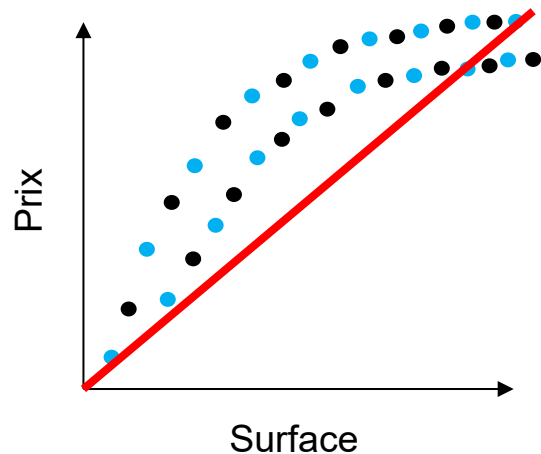


# Sur-entraînement et sous-entraînement

## Sous-entraînement

Erreur sur le jeu d'entraînement : Élevée

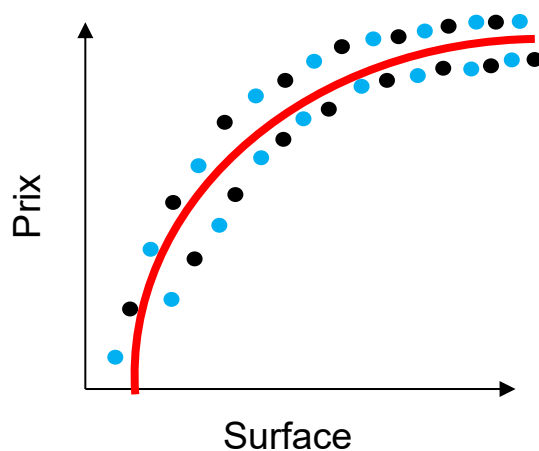
Erreur sur le jeu de test : Élevée



## Entraînement correct

Erreur sur le jeu d'entraînement : Faible

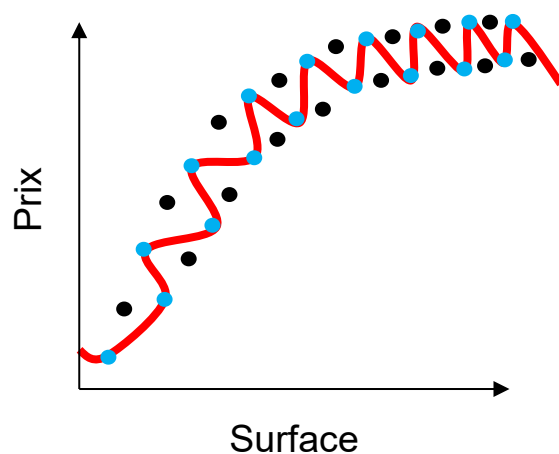
Erreur sur le jeu de test : Faible



## Sur-entraînement

Erreur sur le jeu d'entraînement : Nulle

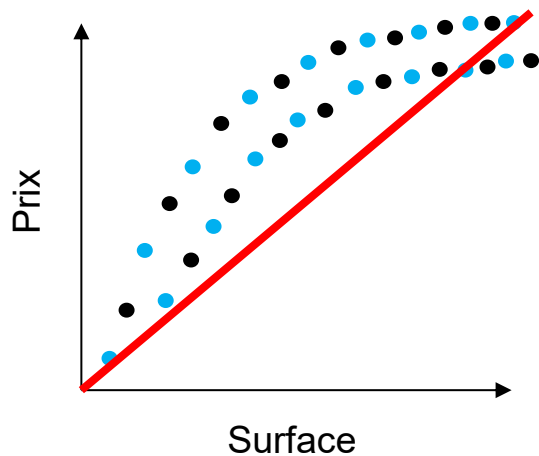
Erreur sur le jeu de test : Moyenne



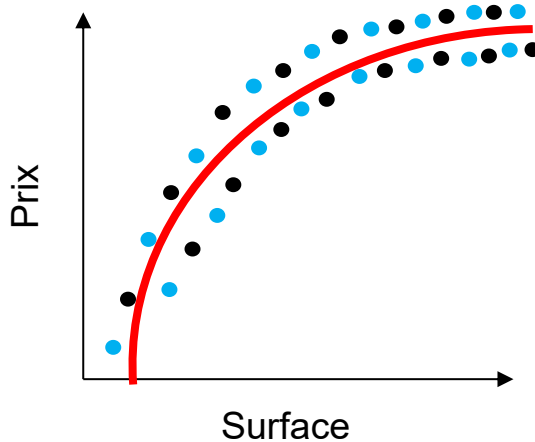


## Complexité du modèle

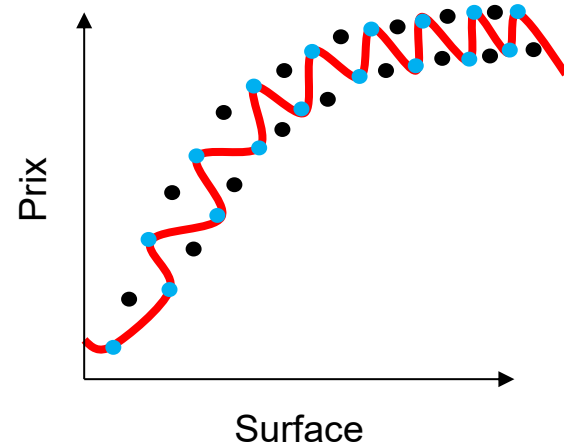
$$\hat{y} = w_0 + w_1 \cdot x_1$$



$$\hat{y} = w_0 + w_1 \cdot x_1 + w_2 \cdot x_1^2$$



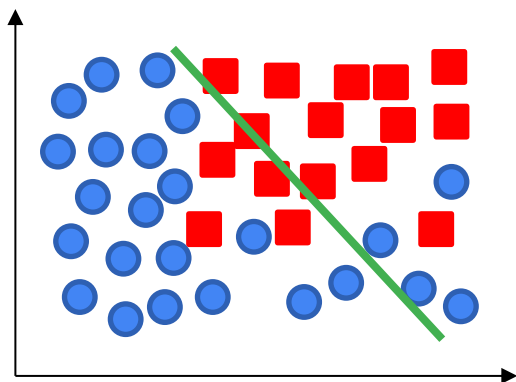
$$\hat{y} = w_0 + w_1 \cdot x_1 + w_2 \cdot x_1^2 + w_3 \cdot x_1^3 + w_4 \cdot x_1^4 + \dots$$



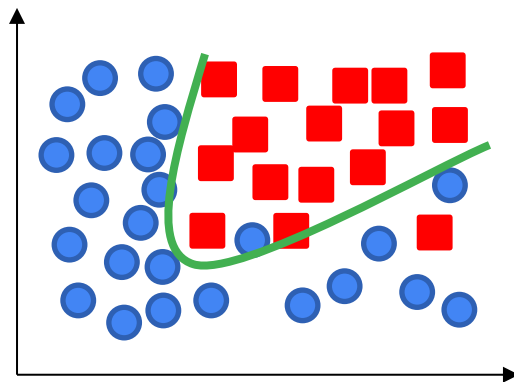
● Jeu d'entraînement   ● Jeu de test



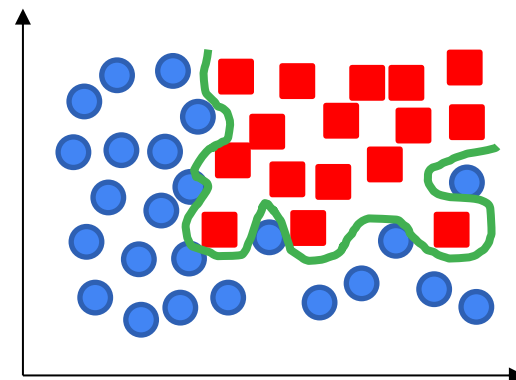
## Pour la classification



Sous-entraînement



Entraînement correct



Sur-entraînement

# La pénalisation L2



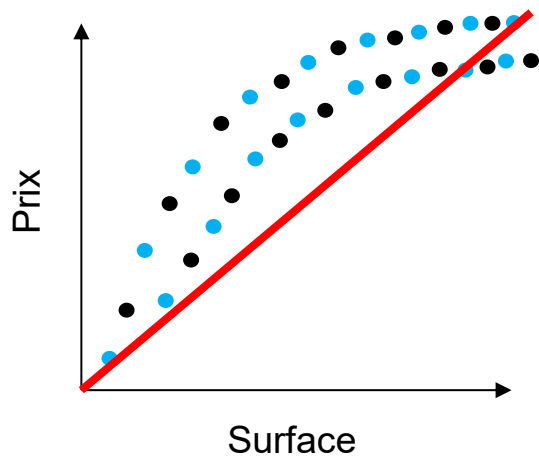
---

Partie 6 : Sur et sous entraînement

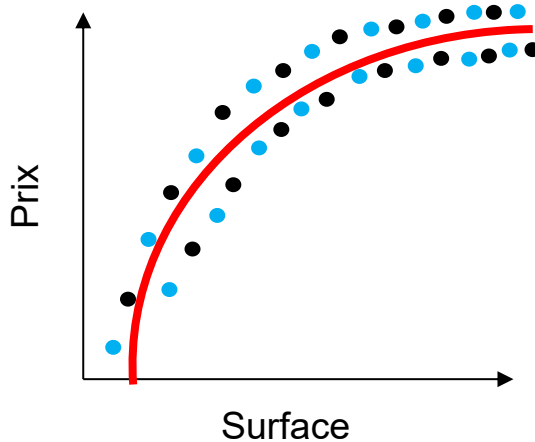


## Complexité du modèle

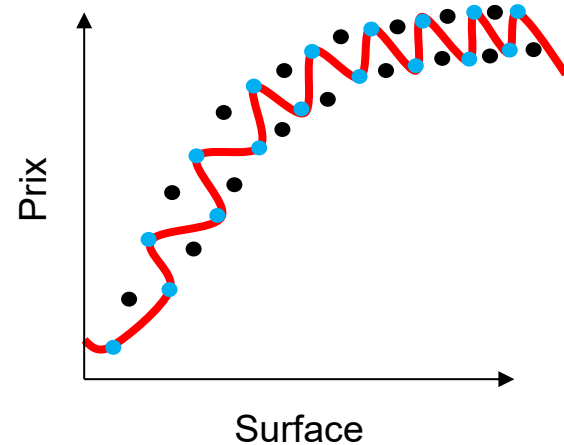
$$\hat{y} = w_0 + w_1 \cdot x_1$$



$$\hat{y} = w_0 + w_1 \cdot x_1 + w_2 \cdot x_1^2$$



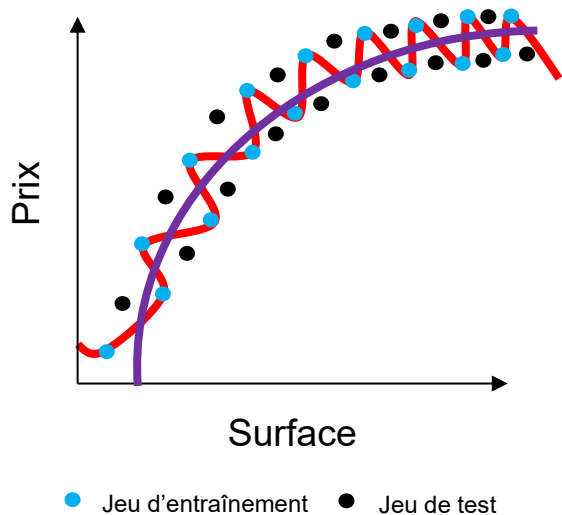
$$\hat{y} = w_0 + w_1 \cdot x_1 + w_2 \cdot x_1^2 + w_3 \cdot x_1^3 + w_4 \cdot x_1^4 + \dots$$



● Jeu d'entraînement ● Jeu de test



## Pénalisation des paramètres



$$\hat{y} = w_0 + w_1 \cdot x_1 + w_2 \cdot x_1^2 + w_3 \cdot x_1^3 + w_4 \cdot x_1^4 + \dots$$

Pénalisation des paramètres

$$\min_w J(w) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2}_{\text{Minimiser l'erreur de prédiction}} + \underbrace{1000 \cdot w_3 + 1000 \cdot w_4 + \dots}_{\text{Minimiser la valeur des paramètres } w_3, w_4, \dots}$$

Minimiser l'erreur  
de prédiction

Minimiser la valeur des  
paramètres  $w_3, w_4, \dots$





## Régression Ridge ou pénalisation L2

- Un modèle avec des paramètres plus homogène est moins sujet au sur-entraînement.

Paramètre de régularisation

$$J(w) = \frac{1}{2m} \left[ \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \sum_{j=1}^n w_j^2 \right]$$

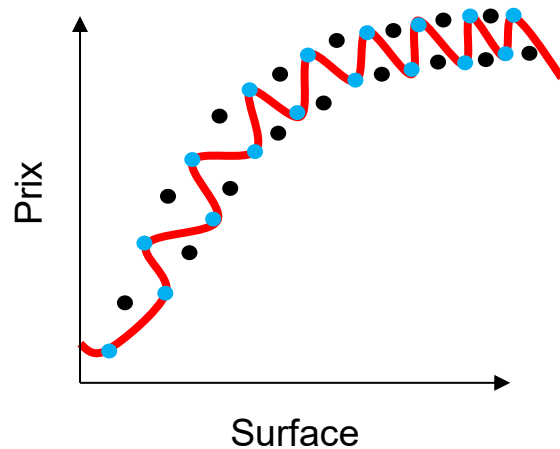
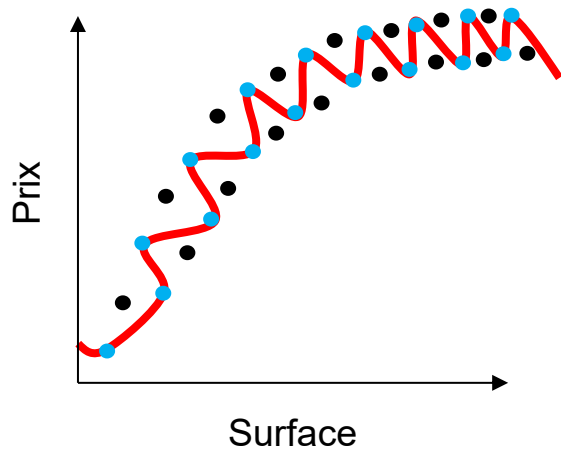
Régularisation



## Impact du coefficient de régularisation

$\lambda$  trop petit

$$J(w) = \frac{1}{2m} \left[ \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \sum_{j=1}^n w_j^2 \right]$$

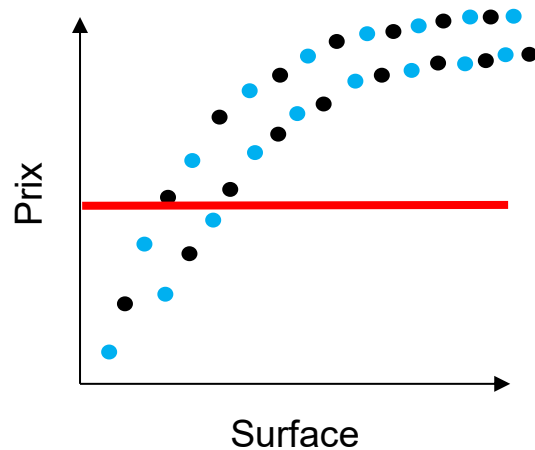
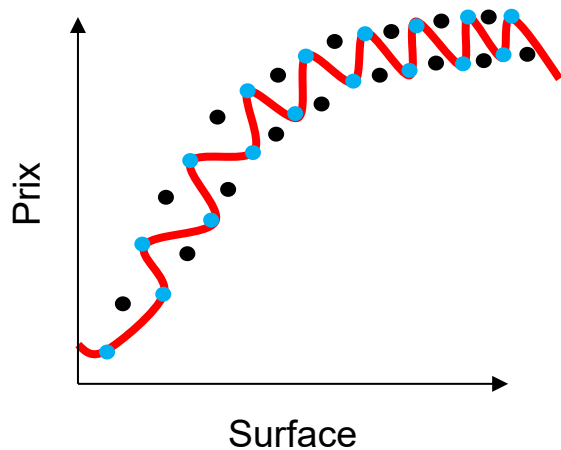




## Impact du coefficient de régularisation

$\lambda$  trop grand

$$J(w) = \frac{1}{2m} \left[ \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \sum_{j=1}^n w_j^2 \right]$$



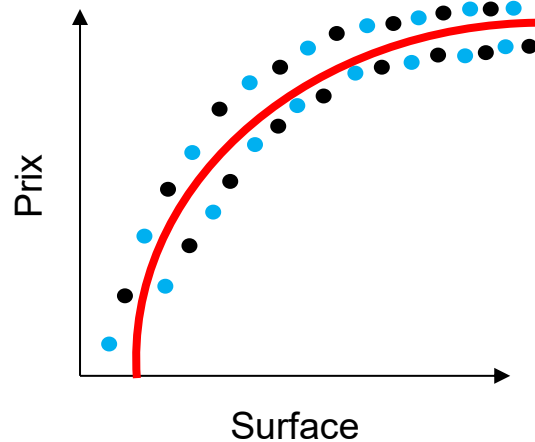
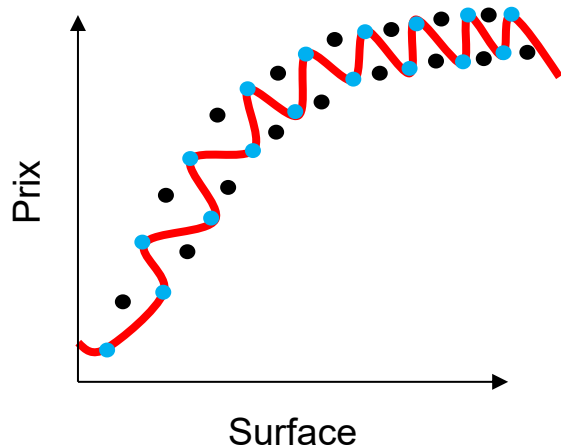


## Impact du coefficient de régularisation

[0.01, ..., 0.1, ..., 0.5]

↖  
 **$\lambda$  adéquat**

$$J(w) = \frac{1}{2m} \left[ \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \sum_{j=1}^n w_j^2 \right]$$



# Le drop out

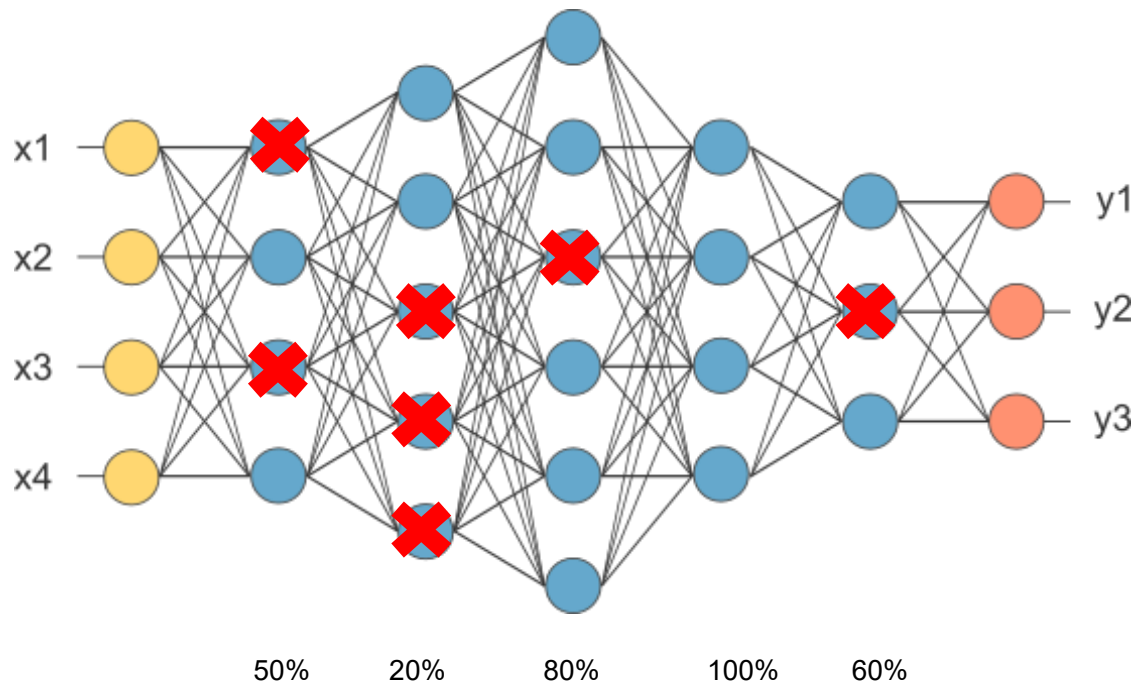


Partie 6 : Sur et sous entraînement



## Le drop out

En utilisant le drop-out, vous ne pouvez pas compter sur une seule fonction, vous devez donc répartir les poids.



# La data augmentation



Partie 6 : Sur et sous entraînement



## La data augmentation



(a) Original



(b) Crop and resize



(c) Crop, resize (and flip)



(d) Color distort. (drop)



(e) Color distort. (jitter)



(f) Rotate  $\{90^\circ, 180^\circ, 270^\circ\}$



(g) Cutout



(h) Gaussian noise



(i) Gaussian blur



(j) Sobel filtering



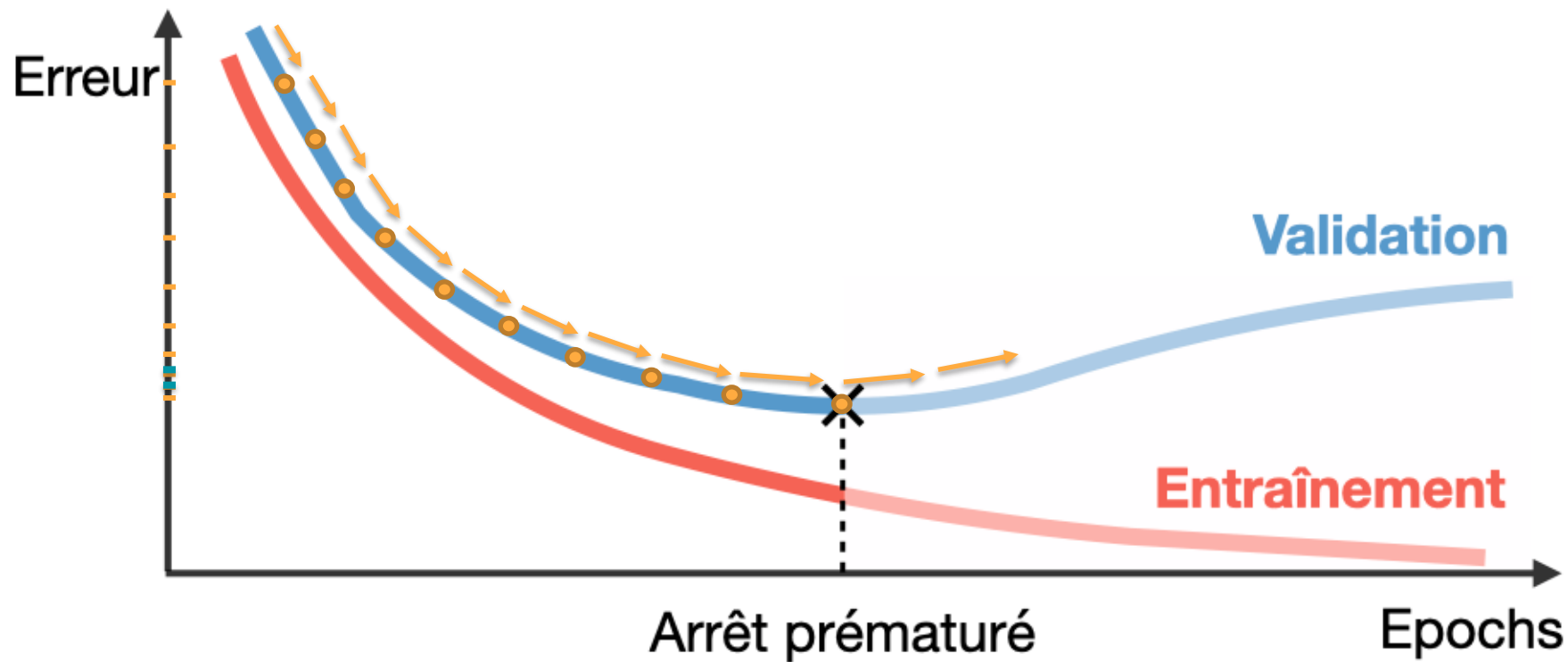
# L'early stopping



Partie 6 : Sur et sous entraînement



## L'early stopping



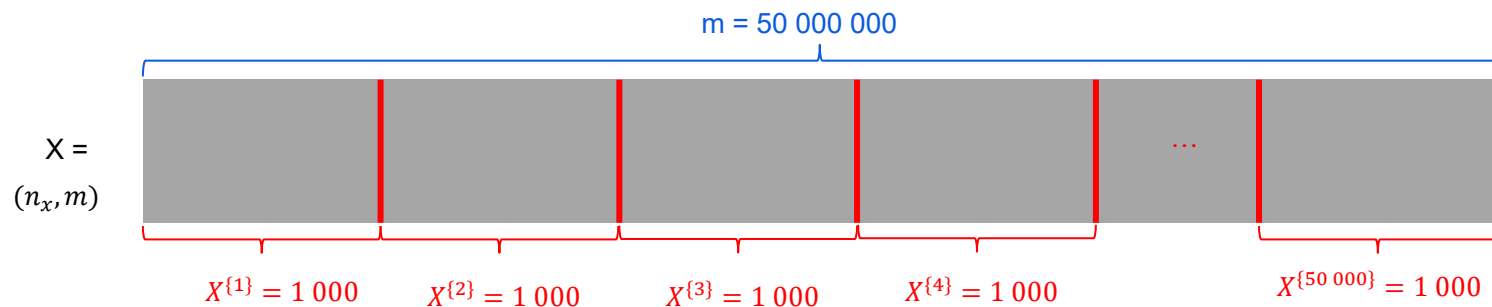
# Mini batch



Partie 6 : Sur et sous entraînement



## Batch vs mini-batch gradient descent



For epoch = 1, ..., 1 000

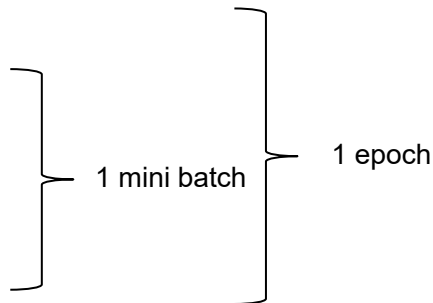
For  $l = 1, \dots, 50\,000$

Forward prop  $X^{\{i\}}$

Calcul du coût  $X^{\{i\}}$

Back prop  $X^{\{i\}}$

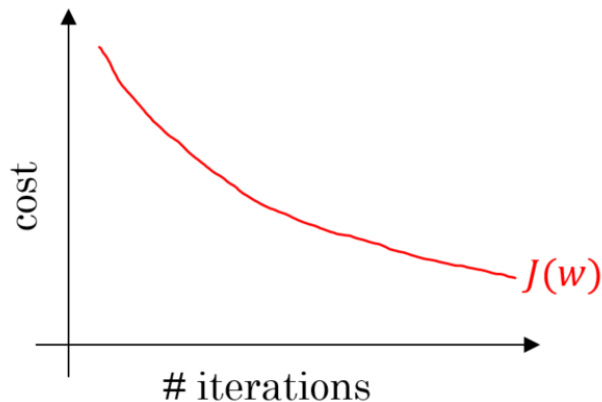
Mise à jour de  $w$  et  $b$





## Batch vs mini-batch gradient descent

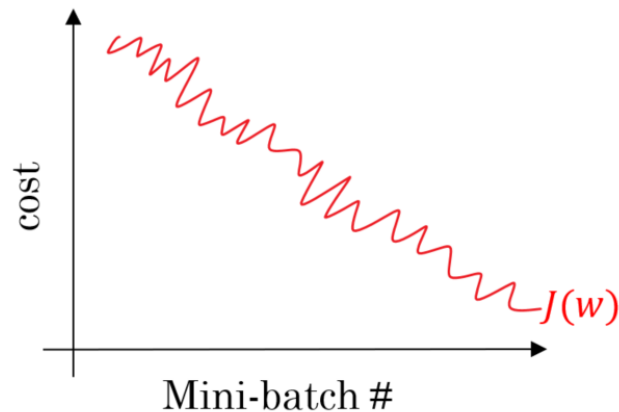
Batch gradient descent



Le batch est trop grand :

Trop long par itération

Mini-batch gradient descent



Le batch est trop petit :

Perte de la vectorisation

Utilisez une puissance de 2 et assurez-vous que votre mini batch correspond à la mémoire de votre GPU.

# Batch normalisation



Partie 6 : Sur et sous entraînement



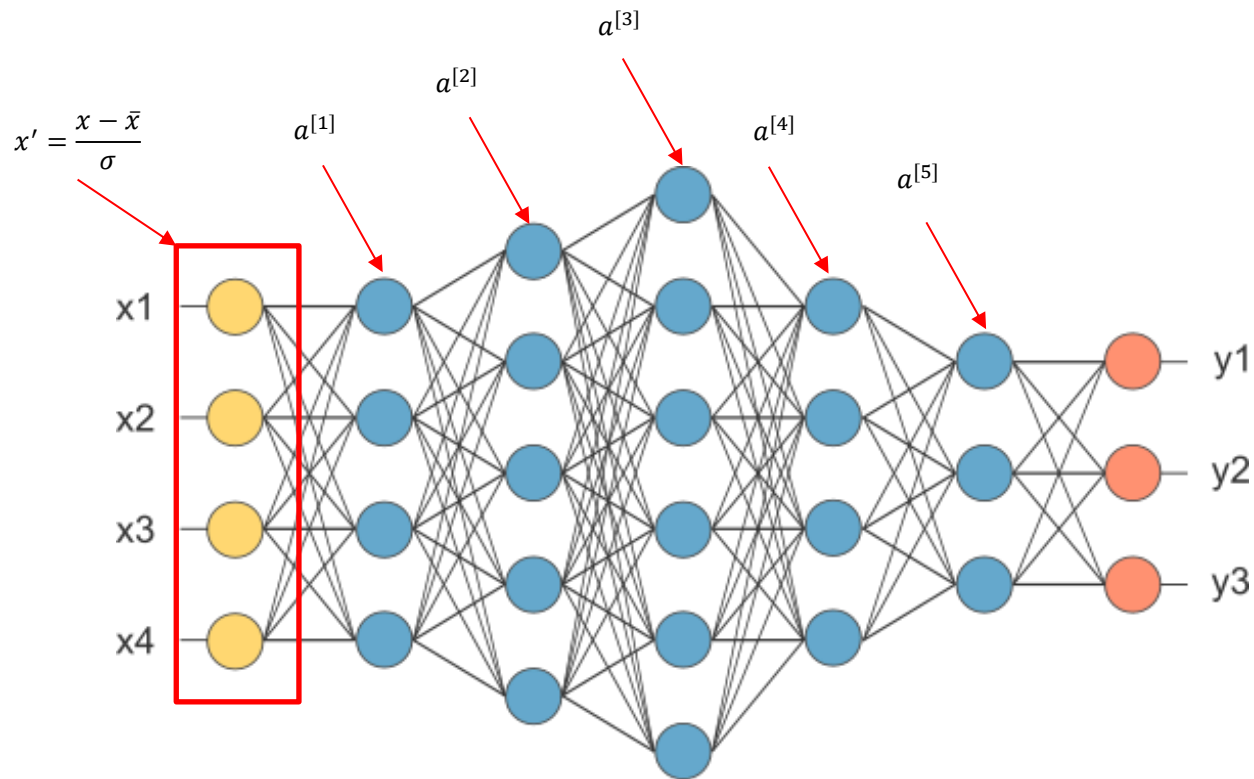
## Normalisation des données d'entrées

$$x' = \frac{x - \bar{x}}{\sigma}$$

Calculer  $\bar{x}$  et  $\sigma$  avec votre ensemble d'entraînement et sauvegardez-les pour les appliquer à l'ensemble de dev et de test.



## Modèle dense







## Disparition et explosion de gradient

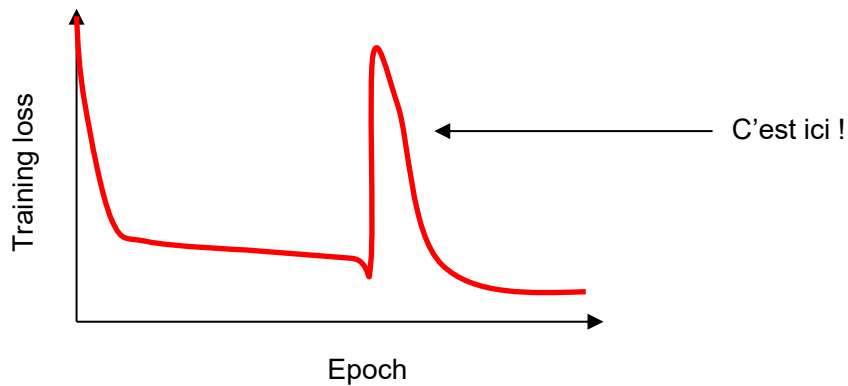
- Lorsque vous construisez un réseau neuronal très profond, votre gradient dans la dernière couche cachée peut être très important ou proche de zéro.
- Il s'agit d'un problème énorme car, avec ce problème, votre modèle ne peut pas apprendre correctement.



# Les explosions de gradient

Les explosions de gradients sont faciles à détecter

Courbe d'apprentissage instable



Les gradients peuvent être trop grands et contenir des NaNs.  
et vous vous retrouvez avec des NaN dans les poids.



# Batch normalization

Pour chaque couche

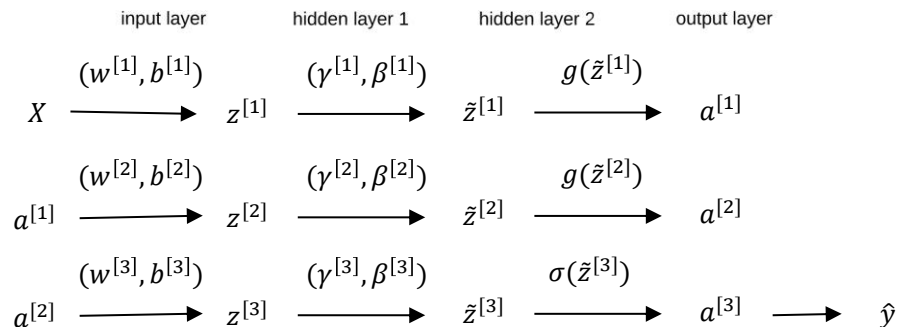
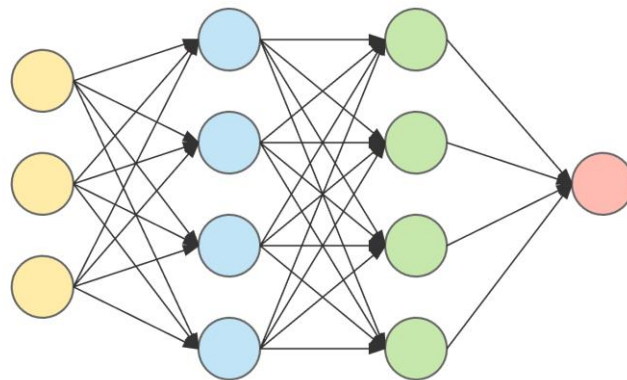
$$\mu^l = \frac{1}{m} \sum_i z_i^l$$

$$\sigma^{l2} = \frac{1}{m} \sum_i (z_i^l - \mu^l)^2$$

$$Z_{norm}^l = \frac{z^l - \mu^l}{\sqrt{\sigma^{l2} - \epsilon}}$$

$$\tilde{Z}^l = \gamma Z_{norm}^l + \beta$$

Couche de  
batch normalisation



# L'entraînement, un processus itératif



Partie 6 : Sur et sous entraînement



## Comment créer son premier modèle ?

Nombre de couches

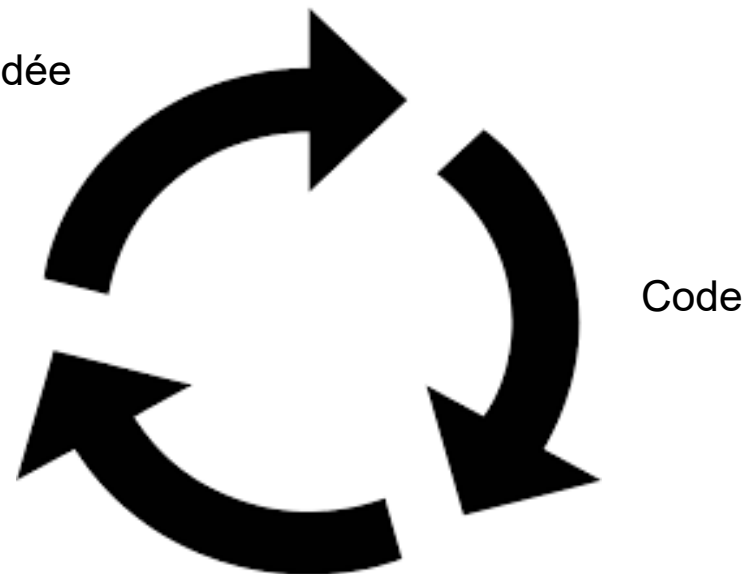
Nombre de neurones

Learning rates

Fonctions d'activations

...

Idée



Code

Résultats



## **Commencer simple**

---

- Tester le modèle le plus simple possible
- Complexifier pour chercher le sur-apprentissage
- Appliquer une régularisation pour chercher la complexité idéale



## Tuning process

- $\alpha$
- Nombre de couches
- Nombre de neurones
- Taille du Mini-batch